

LECTURE 4

Pre-training II: the machine

GPU, step time, memory, and parallel training

Gleb Molodtsov · 28 September 2026



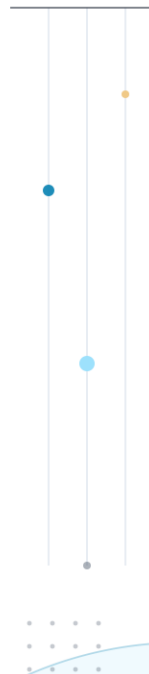
The expensive GPU is idle more often than we think

Poll: what makes one training step slow?

A · matrix multiplications

B · memory traffic

C · GPU communication



Our two levers are steps to target and cost per step

Total time to target $\approx$ **steps to target** $\times$ **seconds per step**

Cost per step is bounded by compute, memory, communication and bubbles.



01

• The machine

SM, Tensor Cores and the memory hierarchy



Processor names describe workloads, not a speed ranking



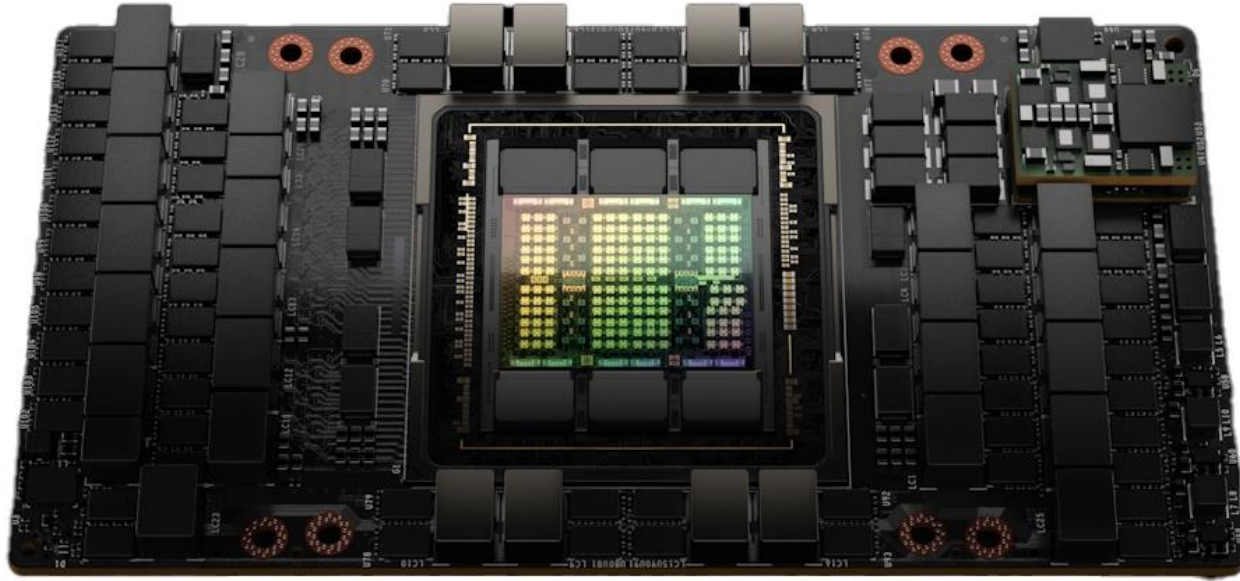
CPU: flexible control and serial work. GPU: throughput over many threads.

NPU/TPU: dedicated tensor accelerator families. Alibaba PPU: T-Head Zhenwu AI accelerator.

For training ask the same questions: matrix throughput, memory, interconnect and software.



The H100 board surrounds compute with HBM and high-speed links



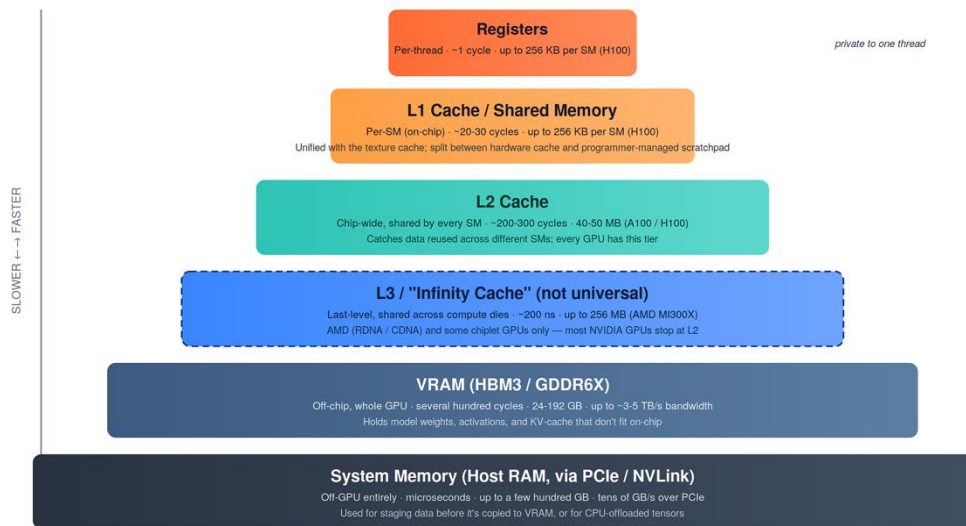
Inside the chip: GPCs contain SMs; L2 serves the whole GPU



Memory levels – catching speed of light

GPU Memory Hierarchy: Registers to System Memory

Faster / smaller memory sits closer to the compute cores; slower / larger memory sits farther away



Figures shown are representative examples (NVIDIA H100 / A100, AMD MI300X) — exact sizes vary by GPU generation and SKU

Original diagram



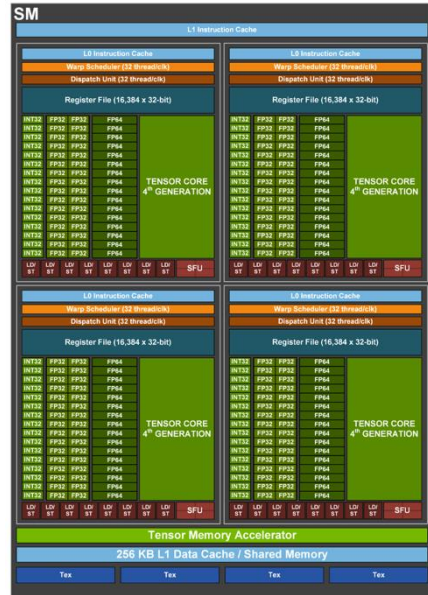
Memory levels – capacity

GPU (Architecture, Year)	L1 / Shared per SM or CU	L2 Cache	L3 / Last-Level Cache
GTX 10-series (Pascal, 2016)	24 KB per SM	256 KB – 4 MB	— none
A100 (Ampere, 2020)	up to 192 KB per SM	40 MB	— none
RTX 4090 (Ada Lovelace, 2022)	~128 KB-class per SM	72 MB	— none
H100 (Hopper, 2022)	up to 256 KB per SM	50 MB	— none
RX 6900 XT (AMD RDNA2, 2020)	per-CU L0/L1	shared L2	128 MB Infinity Cache
MI300X (AMD CDNA3, 2023)	32 KB L1 + 16 KB scalar per CU	4 MB per XCD (×8 dies)	256 MB Infinity Cache

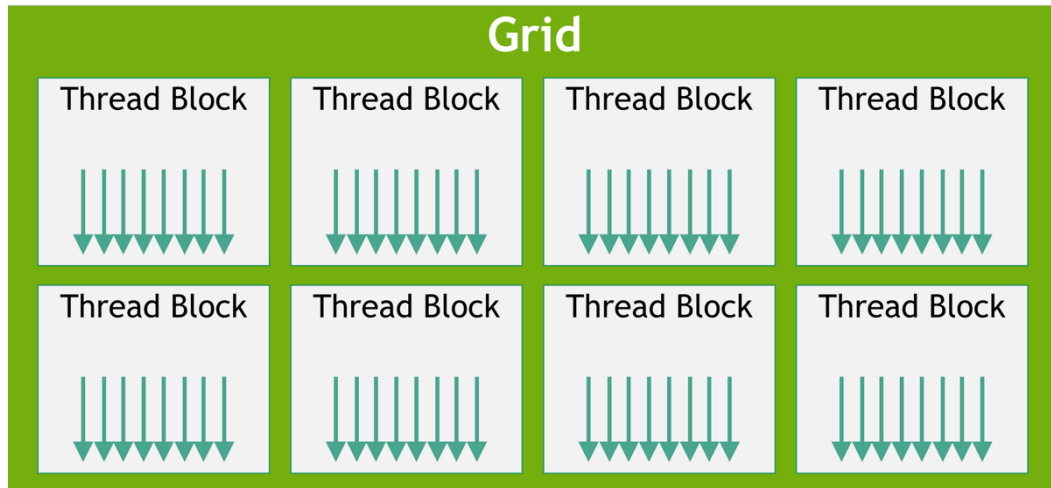


Streaming Multiprocessor schedules warps and holds their registers

Four scheduler partitions, CUDA/Tensor Cores, register files and L1/shared memory in this H100 SM.



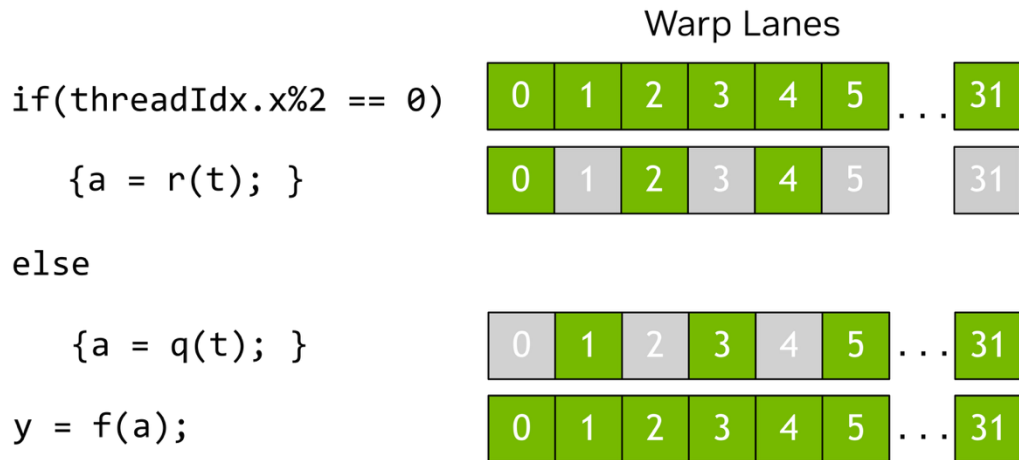
Threads form blocks; blocks are scheduled onto SMs



One kernel launches a grid of blocks. A block owns shared memory; its threads form warps of 32.



SIMT issues a warp instruction; active lanes do different data

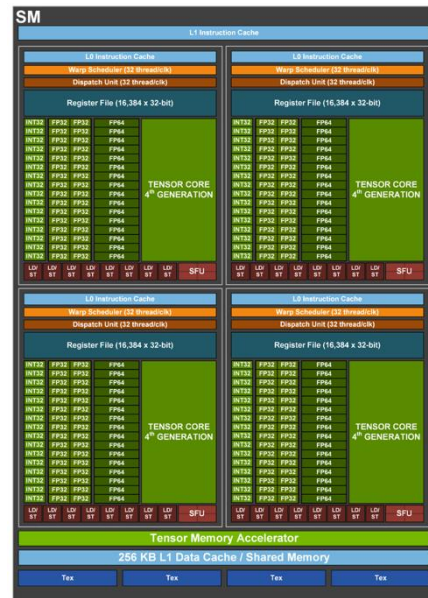


32 lanes share issued instructions. Divergent branches mask lanes; another ready warp may run.



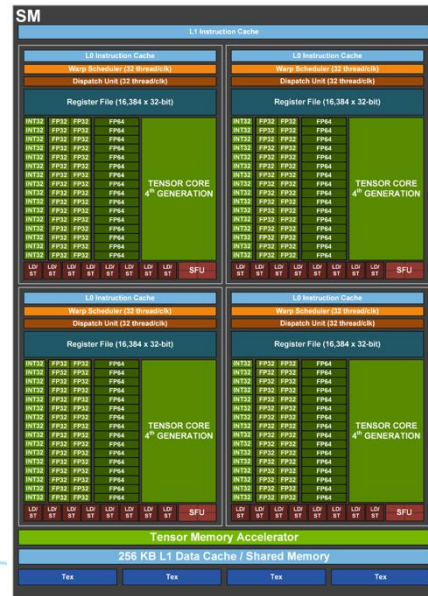
Registers, shared/L1, L2 and HBM differ in scope and control

Registers: per-thread values. Shared: block-managed tiles. L1/L2: caches. HBM: large backing store.



Tensor Cores accelerate matrix tiles; peak FLOPs are not step speed

Large BF16 GEMM uses Tensor Cores. Small shapes, memory stalls and kernel gaps lower achieved speed.



MFU measures useful model arithmetic per GPU second

$\text{MFU} \approx \text{model FLOPs per step} / (\text{peak FLOP/s} \times \text{GPU count} \times \text{wall time}).$

GPU busy %, SM occupancy and MFU answer different questions.



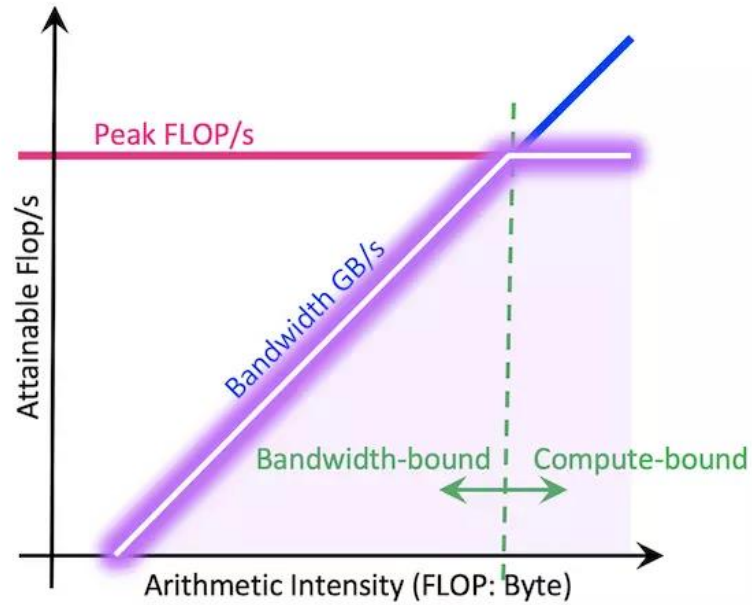
Goodput asks how much allocated time moves the target loss

Scheduled time × runtime progress × program efficiency → useful training work.

A high MFU does not rescue repeated restarts or wasted updates.



Compute bound vs Memory bound



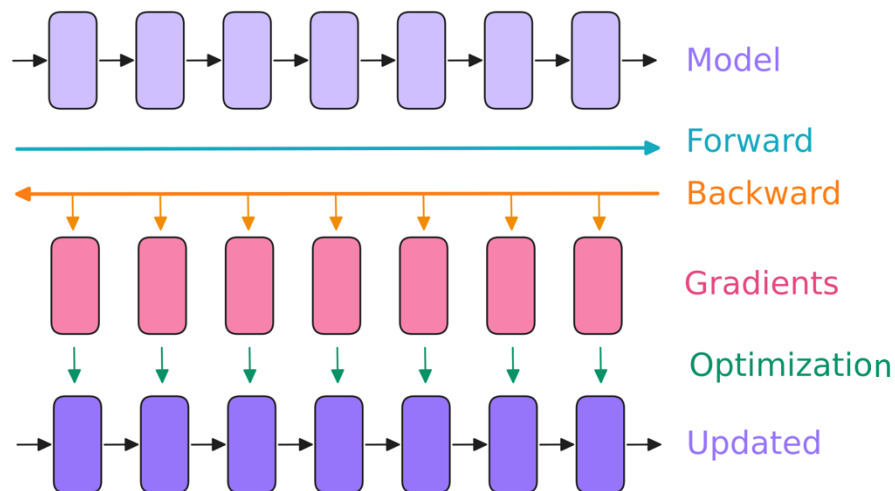
02

• One training step

A published profile, then each Transformer operation



A training step is forward, backward, update—and synchronization

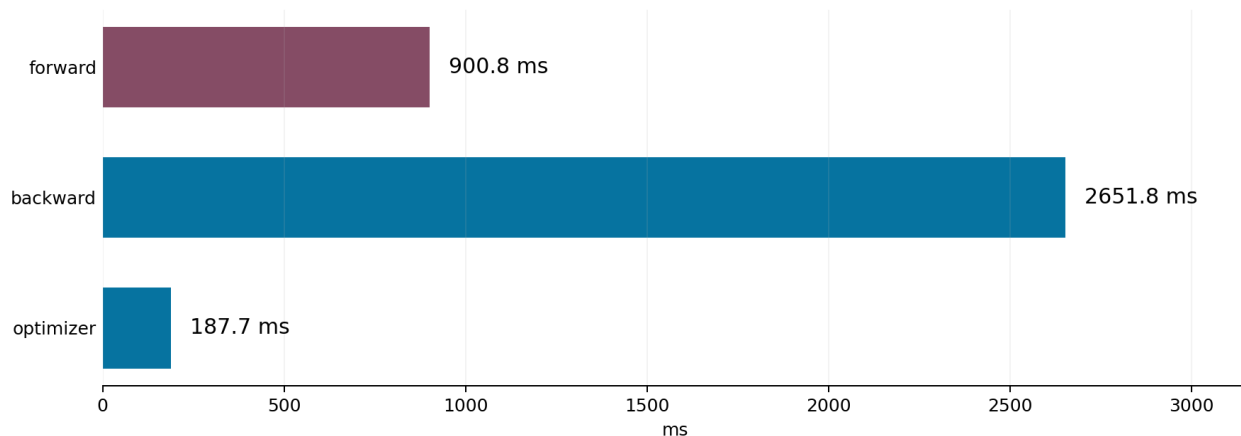


Data staging → forward → loss → backward → gradient reduction → optimizer.

The GPU timeline also shows kernel gaps, transfers and overlap.



7B step: backward dominates, optimizer is small

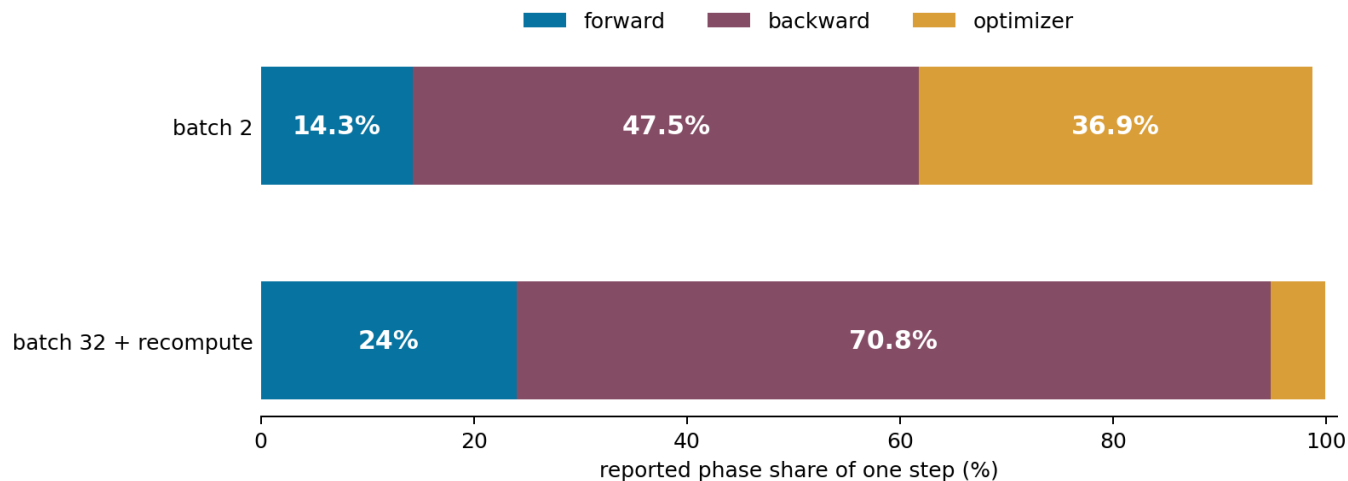


A800, batch 32 with recomputation: forward 900.8 ms; backward 2651.8 ms; optimizer 187.7 ms.

Shares: 24.0% / 70.8% / 5.1%.



The same optimizer becomes 37% at batch 2, 5% at batch 32



On A800, parameter update time barely changes; forward/backward grow.

The batch-32 run also uses recomputation: this is not an isolated batch ablation.



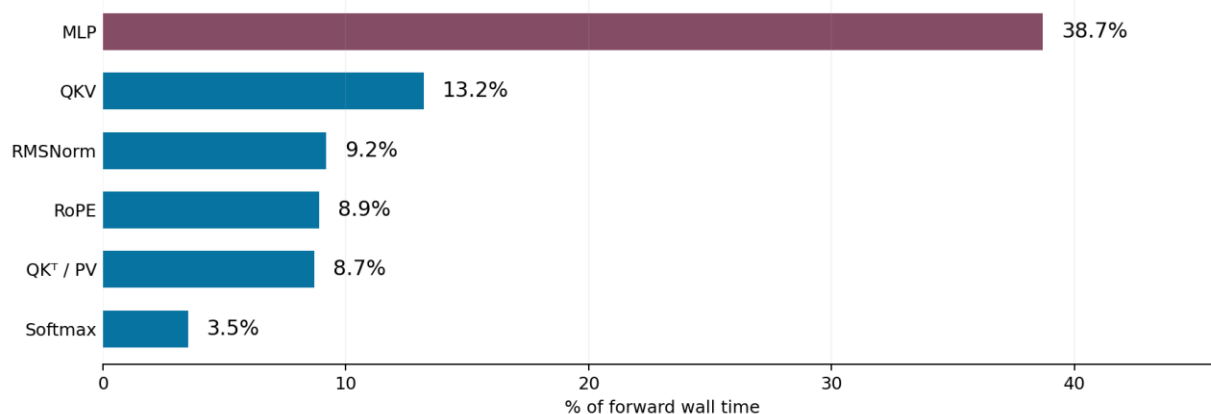
Predict the largest decoder submodule in the published trace

Vote: QKV / attention matrix products / RoPE / MLP / RMSNorm.

Can a small FLOP count still consume a noticeable fraction of time?



At short context, dense MLP wins this measured forward



A800, Llama 2 7B at batch 2: MLP 38.7%, QKV 13.2%,
score/value matmuls 8.7% of forward.



Embedding lookup moves rows; it barely multiplies numbers

Input embedding: indexed reads, sparse access and HBM/cache behavior.

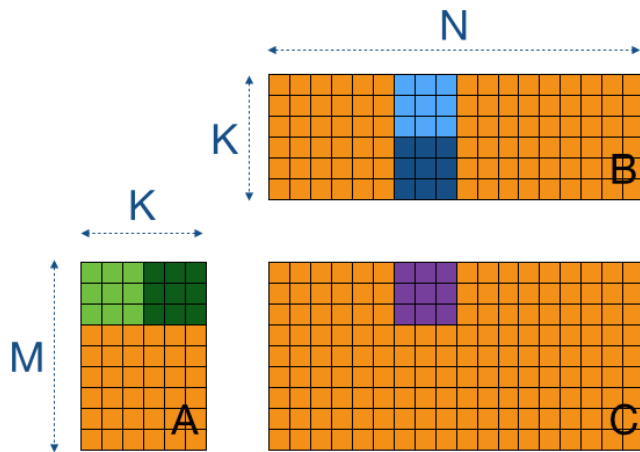
Its time is often small relative to blocks; activation size scales with $B \times S \times H$.



QKV and output projections are GEMMs with shape-dependent speed

FLOPs roughly scale with batch tokens $\times$ hidden².

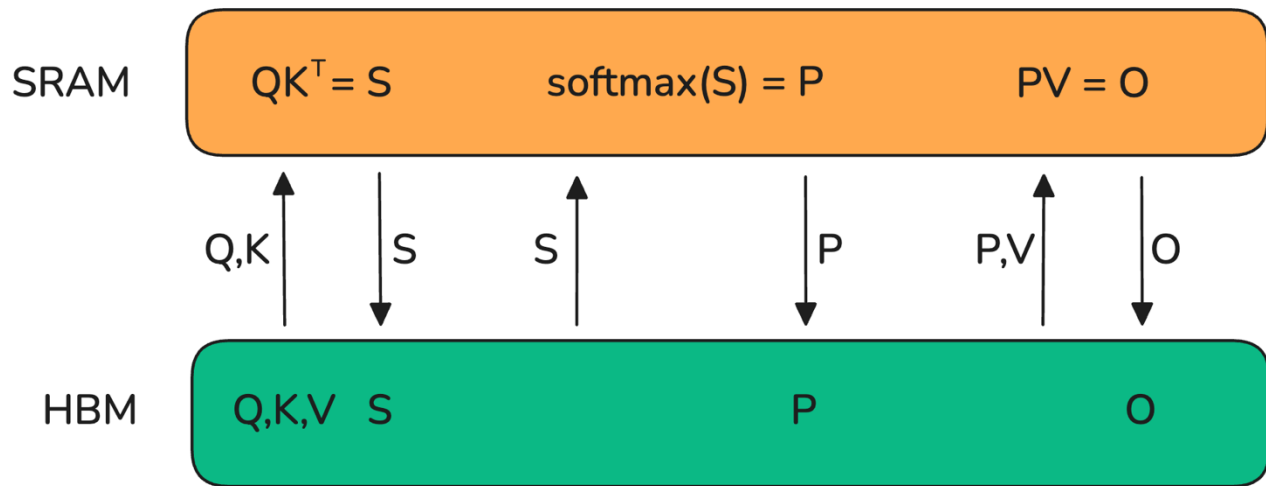
At small microbatch, the same matrices become skinny GEMMs with worse utilization.



$$\begin{bmatrix} \text{purple} \end{bmatrix} = \begin{bmatrix} \text{green} \end{bmatrix} \times \begin{bmatrix} \text{light blue} \end{bmatrix} + \begin{bmatrix} \text{dark green} \end{bmatrix} \times \begin{bmatrix} \text{dark blue} \end{bmatrix}$$



Attention grows with S^2 ; short-context profiles hide the pain



Attention FLOPs $\propto B S^2 H$; MLP FLOPs $\propto B S H^2$.

Their ratio grows roughly as S/H ; FlashAttention changes IO, not the quadratic arithmetic.



FlashAttention keeps score tiles near compute

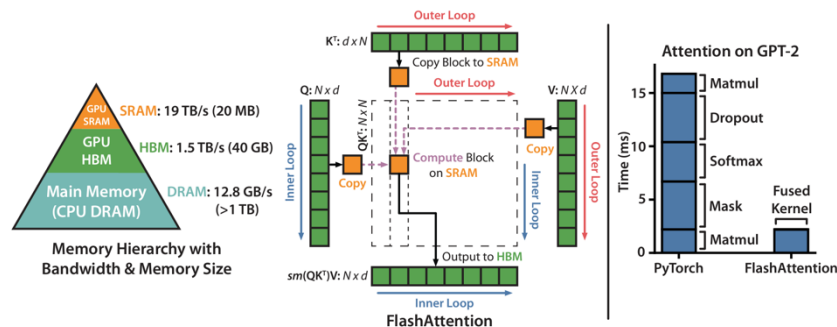


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of Q matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an $7.6\times$ speedup on the attention computation.

Tiling and online softmax avoid materializing the $S \times S$ score matrix in HBM.

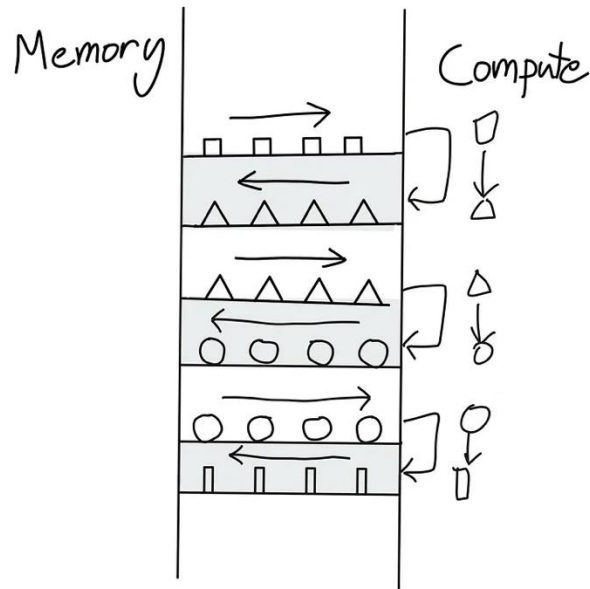
Exact attention, different IO pattern; speedup depends on shapes and implementation.



RoPE, RMSNorm and activation functions can be memory-bound

Each pass reads/writes tensors but does little arithmetic per byte.

Small kernels and repeated launches add overhead; fusion saves intermediate traffic.



The MLP is usually the largest dense compute block

Gate/up/down projections are GEMMs; SwiGLU adds a smaller elementwise stage.

The matrix shape and microbatch decide achieved throughput.



The LM head projects every token into the vocabulary



Logits cost $\approx 2 \times B \times S \times H \times V$ forward FLOPs.

Large vocabulary or untied output projection can make the tail visible; fused loss avoids retaining full logits.

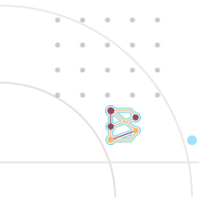


The optimizer is light on FLOPs but heavy on bytes



AdamW reads/writes weights, gradients and moment states; often bandwidth-bound.

Its share of wall time depends on implementation, sharding and the size of forward/backward.



Compute-bound or memory-bound depends on $B \cdot S$, H and V

QKV/MLP: GEMM, often compute-bound when $B \cdot S$ and H are large.

Embedding, norms, RoPE, AdamW: low arithmetic intensity; usually HBM/launch-bound.



Attention and LM head cross over at different shapes

Attention: S^2 compute and (naïvely) S^2 scores; long S changes the ranking.

LM head: $B \cdot S \cdot H \cdot V$ GEMM and $B \cdot S \cdot V$ logits; large V exposes it.



Inference has a different bottleneck map



Prefill uses larger GEMMs and attention; decode repeatedly reads weights and KV cache for few new tokens.

We return to serving, batching and quantization in lectures 11–12.



03

Memory through time

Weights, states, activations and the peak



Memory for components

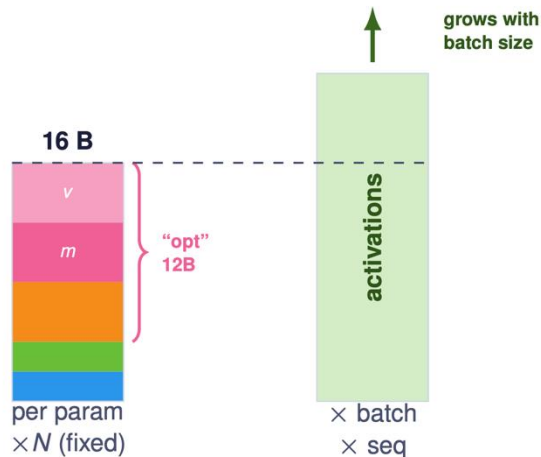
Four buckets scale with parameters N :

- **Weights** (BF16) — 2 B
- **Gradients** (BF16) — 2 B
- **FP32 master weights** — 4 B
- **Optimizer state** m, v — 8 B

⇒ AdamW = **16 B/param**, 12 of them the optimizer's.

- **Activations** — a fifth consumer on a *different axis*: scales with **batch** × **sequence**, not N .

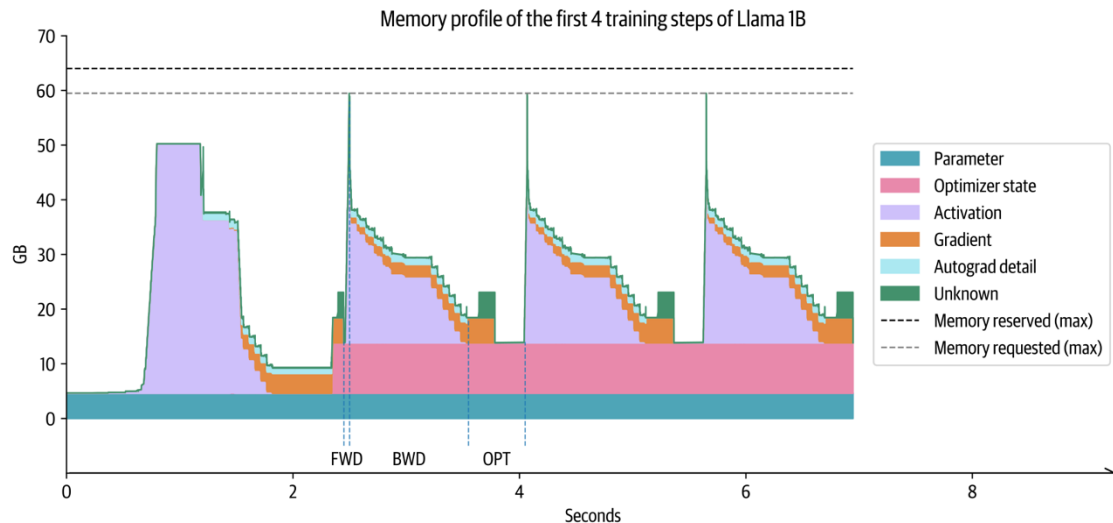
16 B/param is what optimizer tricks attack — but the memory **peak** is set by activations, i.e. by your **batch size**.



Two axes: per-parameter (fixed) vs activations (you tune via batch size).



The peak matters: activations rise and fall through the step

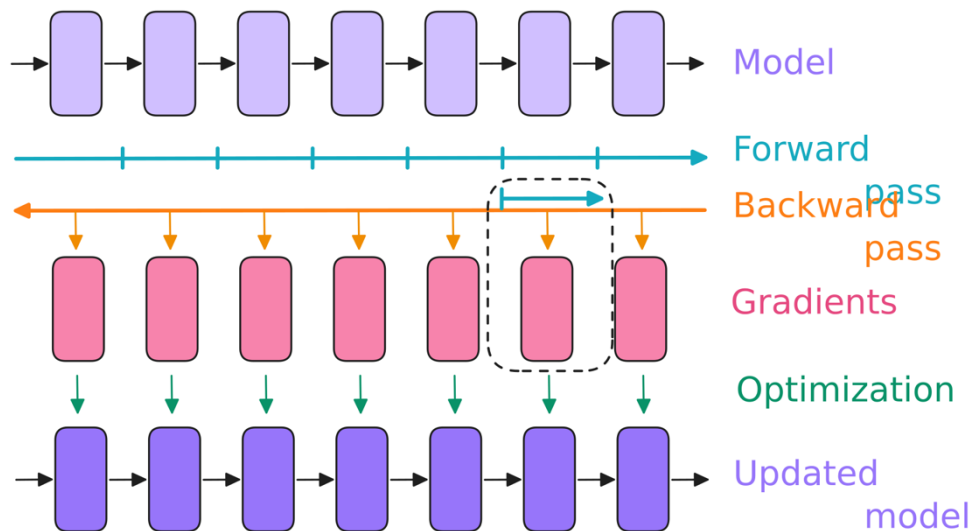


Weights and optimizer states persist. Forward saves activations; backward consumes them and creates gradients.

Peak allocated/reserved memory depends on microbatch, sequence length and checkpoints.



Activation checkpointing trades recomputation for HBM



Save fewer intermediate activations, rerun selected forward ops in backward.

Use when activation memory limits microbatch or context; wall time rises.



Choose an optimization by the resource it actually releases



GEMM → tensor-core utilization/precision. 1D kernels → fusion, fewer bytes.

Activations → FlashAttention/checkpointing. Persistent states → ZeRO.

Network stalls → overlap, topology, parallel layout.



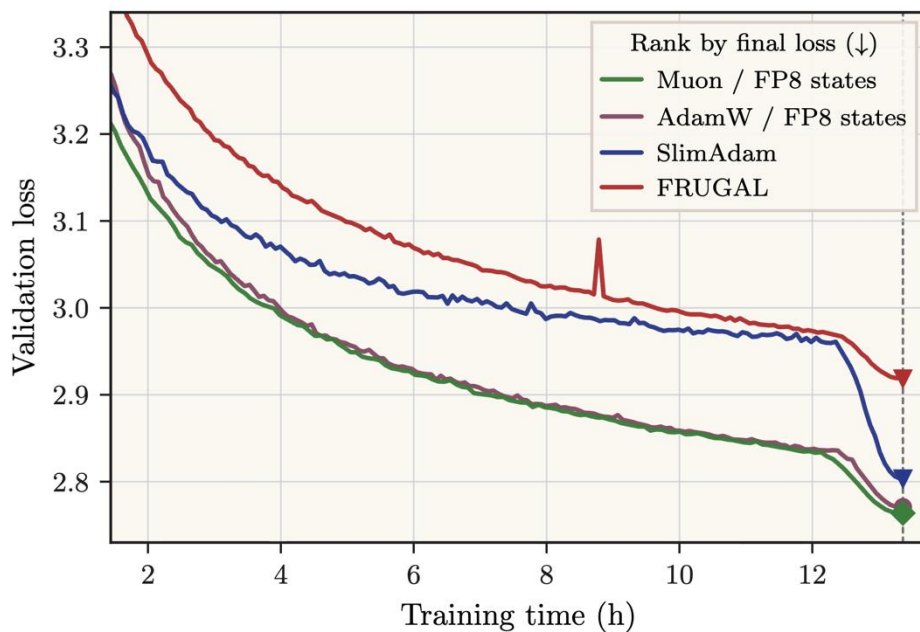
04

Time to target

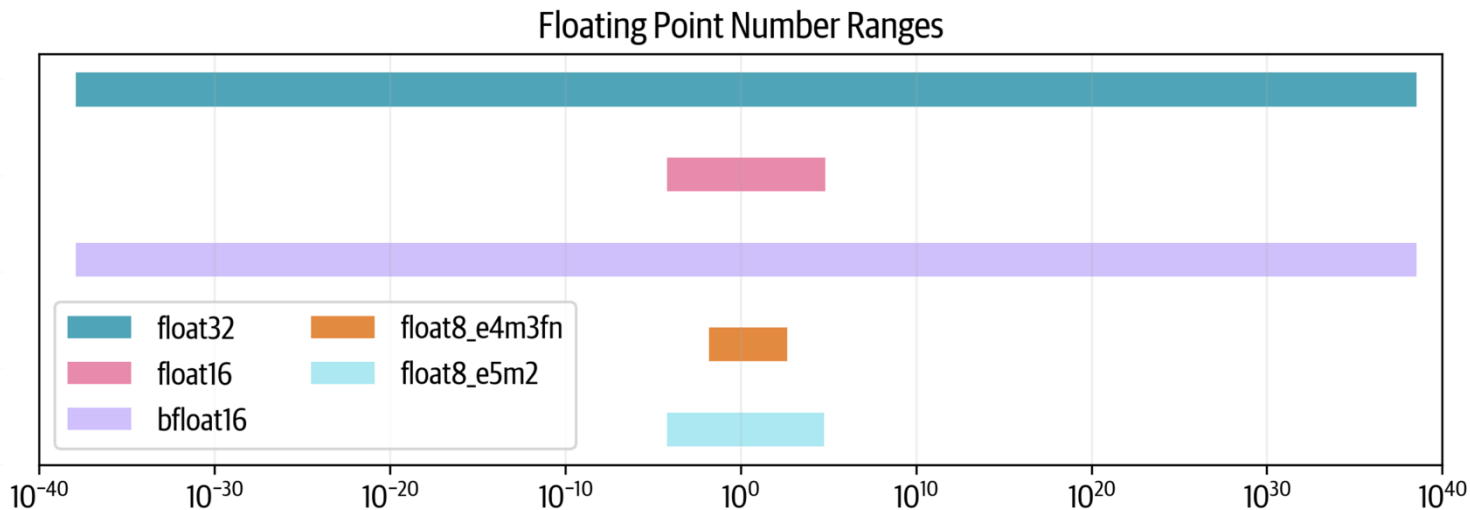
Convergence and arithmetic precision



Time to target couples convergence with step cost



Quantize tensors separately: compute, states and communication



BF16/FP8 matrix inputs affect Tensor Core speed; low-bit moments affect state memory.

Gradient communication and activations have separate numerical constraints.

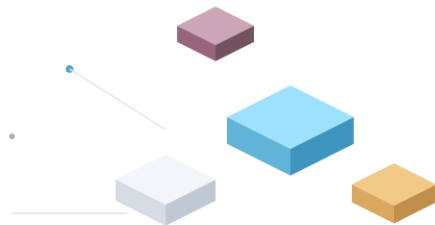
NVIDIA Hopper Transformer Engine; Dettmers et al. (2022)



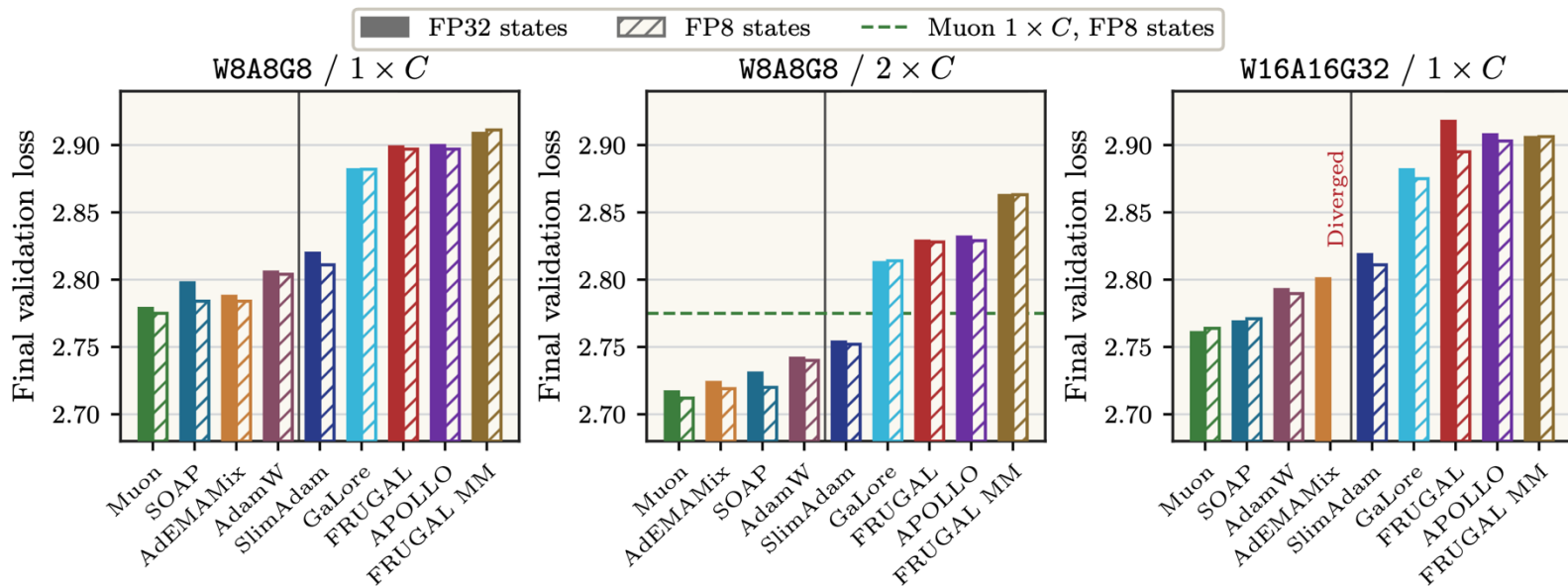
Optimizer state can dominate persistent memory

AdamW moments: two FP32 tensors (8 bytes/parameter), plus possible FP32 master weights.

For a 7B recipe, state layout decides whether ~56 or ~84 GB of extras persist.



Memory-efficient optimizers trade state bytes for math or approximation



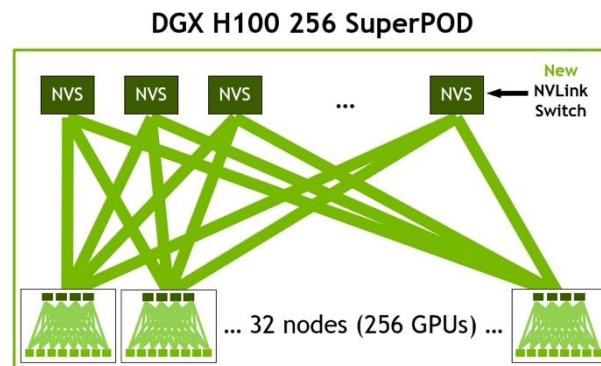
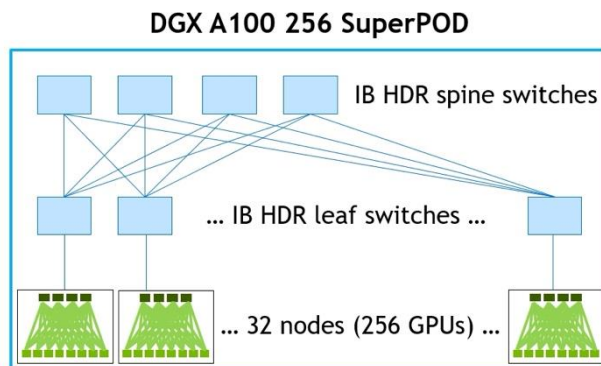
05

Beyond one GPU

Links and collective communication



A cluster adds NVLink islands and a slower network tier



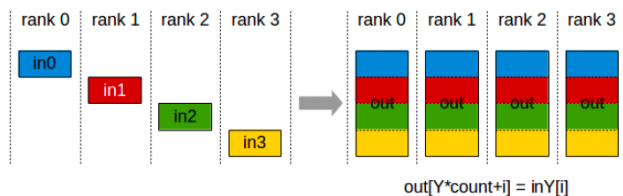
Fully NVLink-connected
Massive bisection bandwidth

	A100 SuperPod			H100 SuperPod			Speedup	
	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Bisection	Reduce
1 DGX / 8 GPUs	2.5	2,400	150	16	3,600	450	1.5x	3x
32 DGXs / 256 GPUs	80	6,400	100	512	57,600	450	9x	4.5x

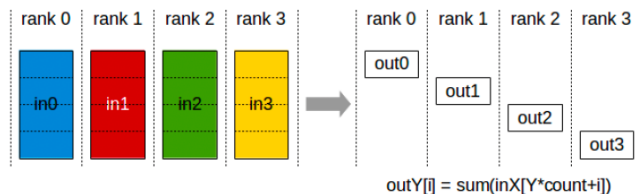


All-gather rebuilds; reduce-scatter sums and shards

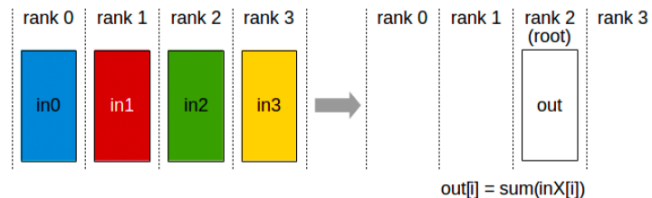
All Gather



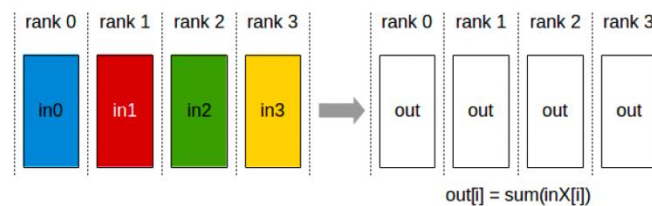
Reduce Scatter



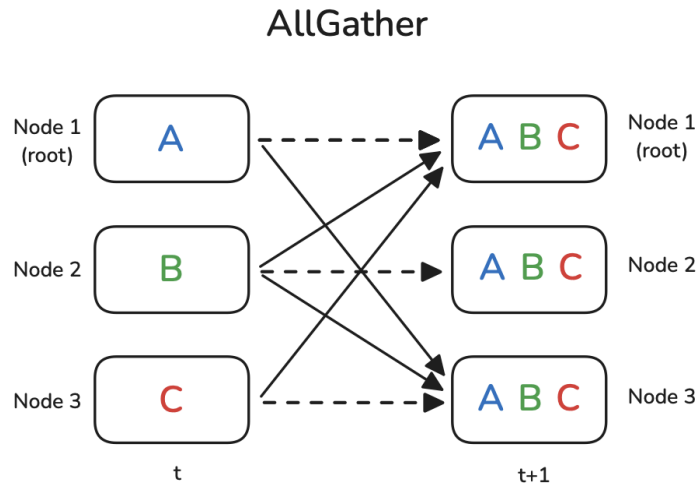
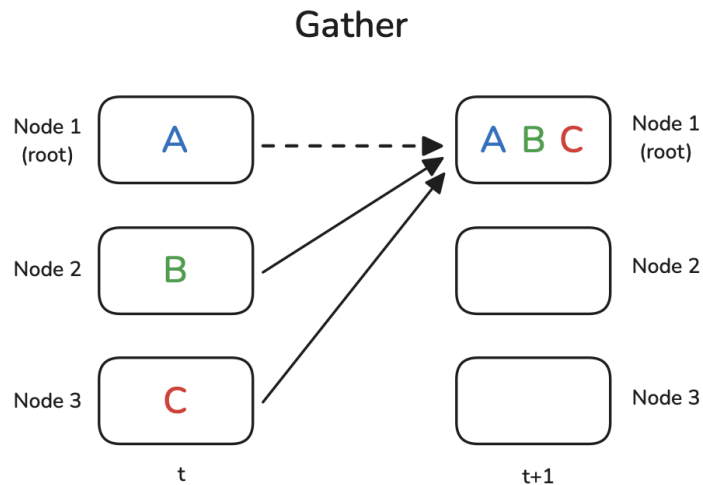
Reduce



All reduce



Collectives have different meanings and communication volumes



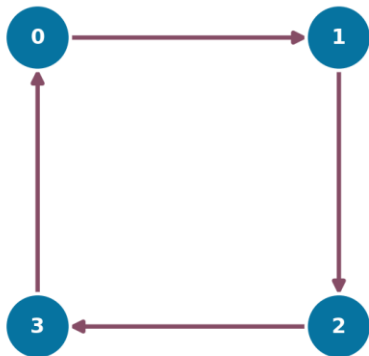
All-reduce sums gradients. All-gather reconstructs a shard. Reduce-scatter sums and leaves shards.

The same logical bytes travel with different topology and overlap.

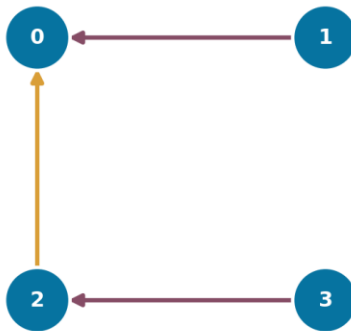


Ring, tree and butterfly are schedules for collectives

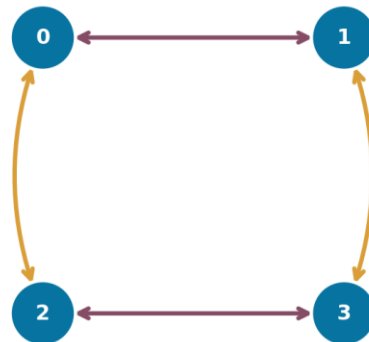
Ring



Tree reduction



Recursive doubling



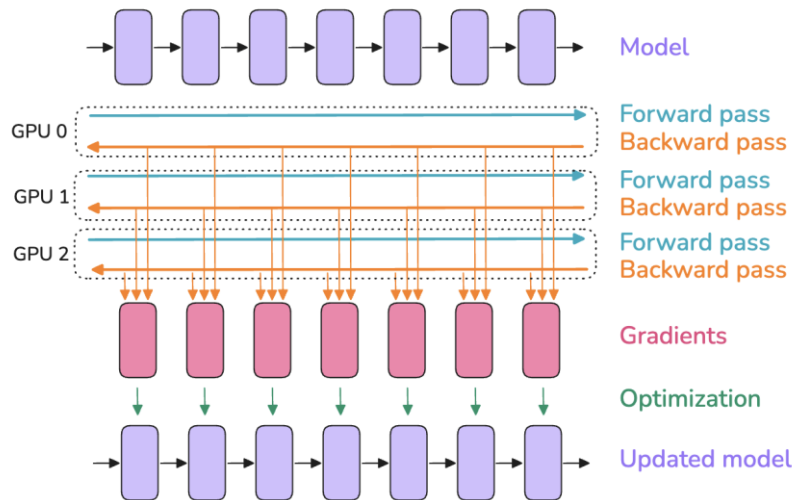
06

• Data parallelism

Replicas, gradients and ZeRO



Data parallelism duplicates model, splits batches

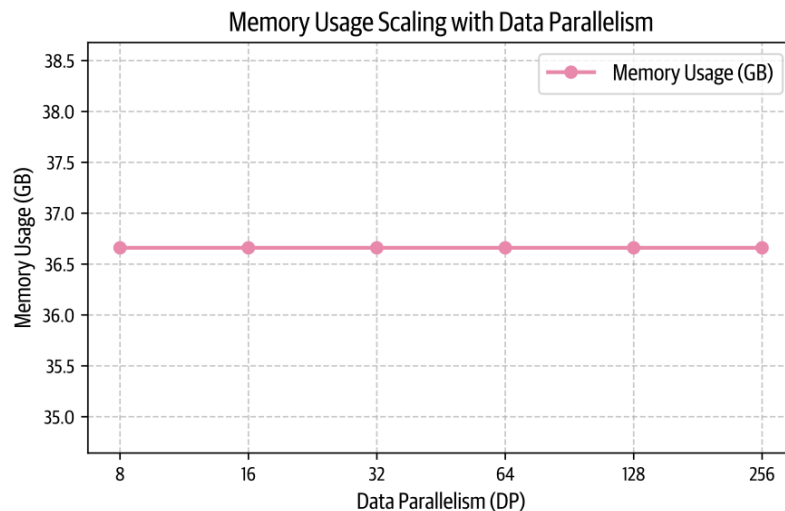
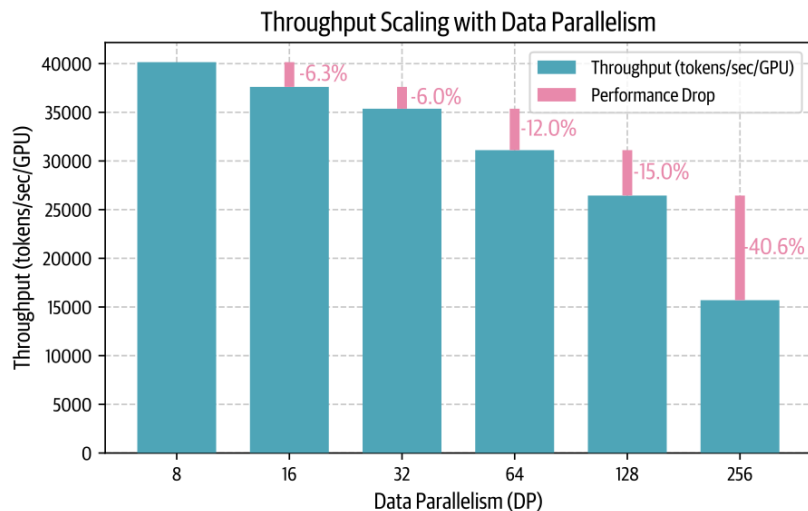


Every GPU runs its own microbatch, then gradients are synchronized.

Global batch = per-GPU microbatch × accumulation × DP degree.



DP speeds throughput while each replica must still fit



Compute scales across independent batches; gradient all-reduce and stragglers cut efficiency.

Accumulation reduces synchronization frequency, but enlarges effective batch.



Quick exercise: what fits on eight 80 GB GPUs?

7B parameters; illustrative AdamW persistent state ≈ 112 GB per replica.

Does plain DP fit? What would you shard first?



Memory for components

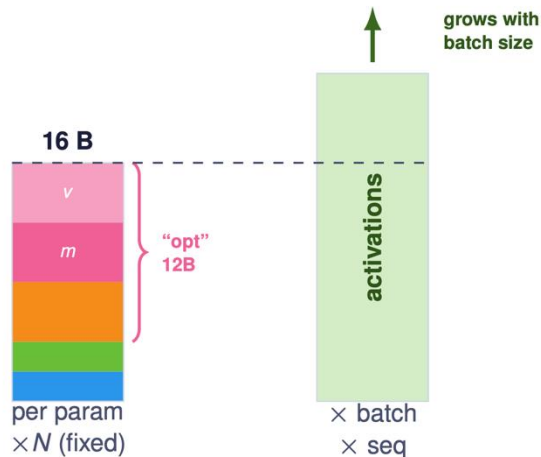
Four buckets scale with parameters N :

- **Weights** (BF16) — 2 B
- **Gradients** (BF16) — 2 B
- **FP32 master weights** — 4 B
- **Optimizer state** m, v — 8 B

⇒ AdamW = **16 B/param**, 12 of them the optimizer's.

- **Activations** — a fifth consumer on a *different axis*: scales with **batch** × **sequence**, not N .

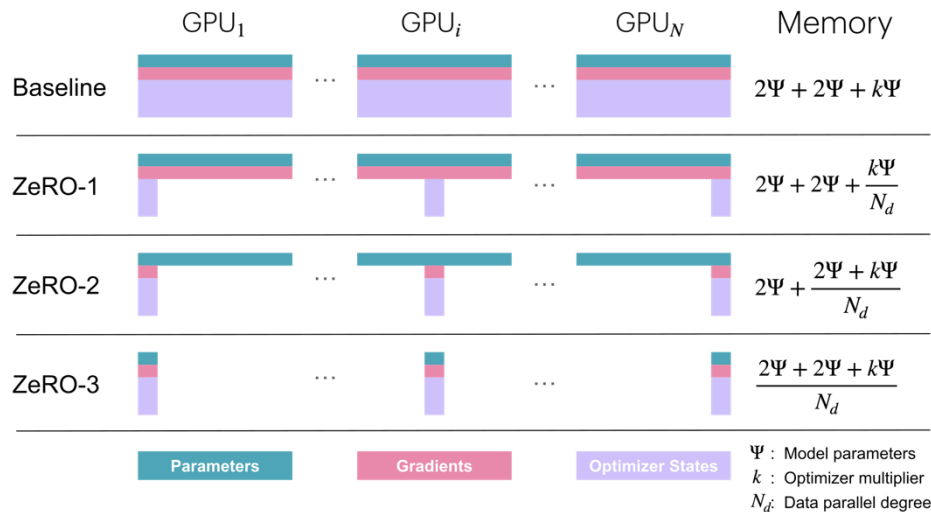
16 B/param is what optimizer tricks attack — but the memory **peak** is set by activations, i.e. by your **batch size**.



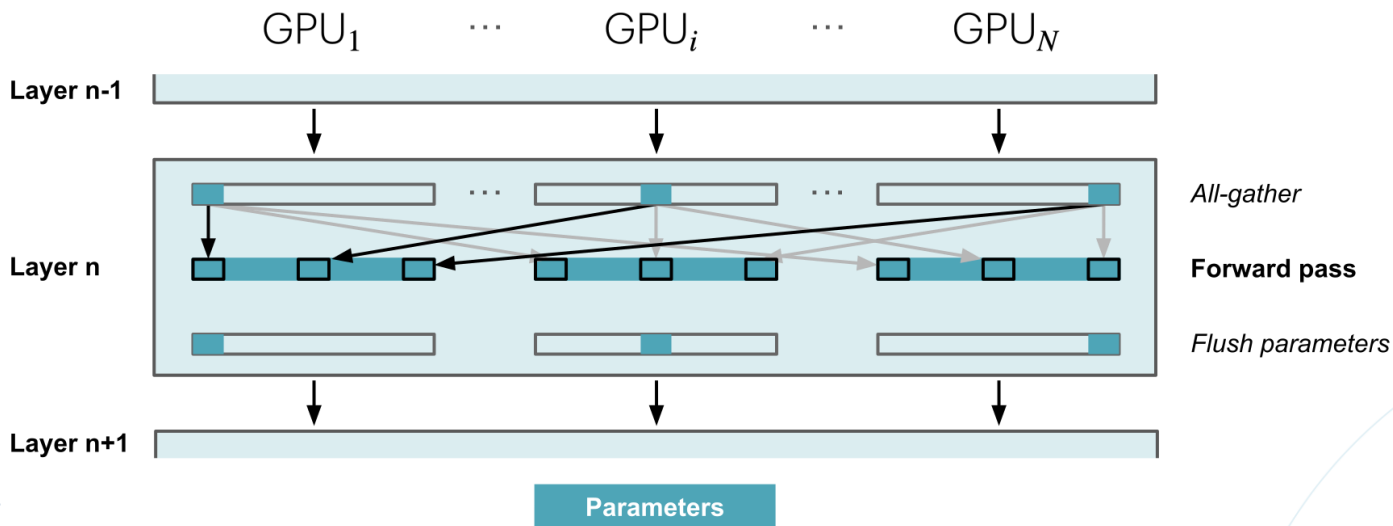
Two axes: per-parameter (fixed) vs activations (you tune via batch size).



ZeRO-1 shards optimizer states; ZeRO-2 also shards gradients



ZeRO-3 shards weights too -- and must gather them to compute



Parameters, gradients and optimizer states are partitioned across DP ranks.

Layerwise all-gather buys memory at the cost of bandwidth, latency and prefetch complexity.



07

Tensor parallelism

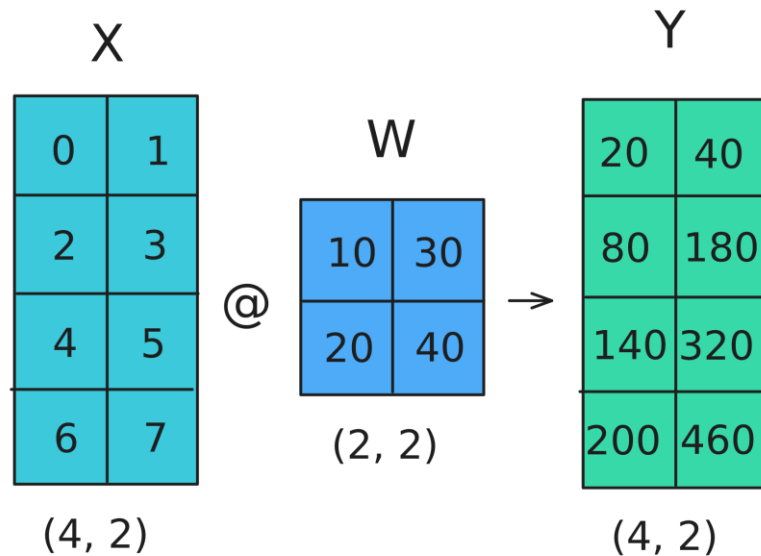
Split the matrix, communicate activations



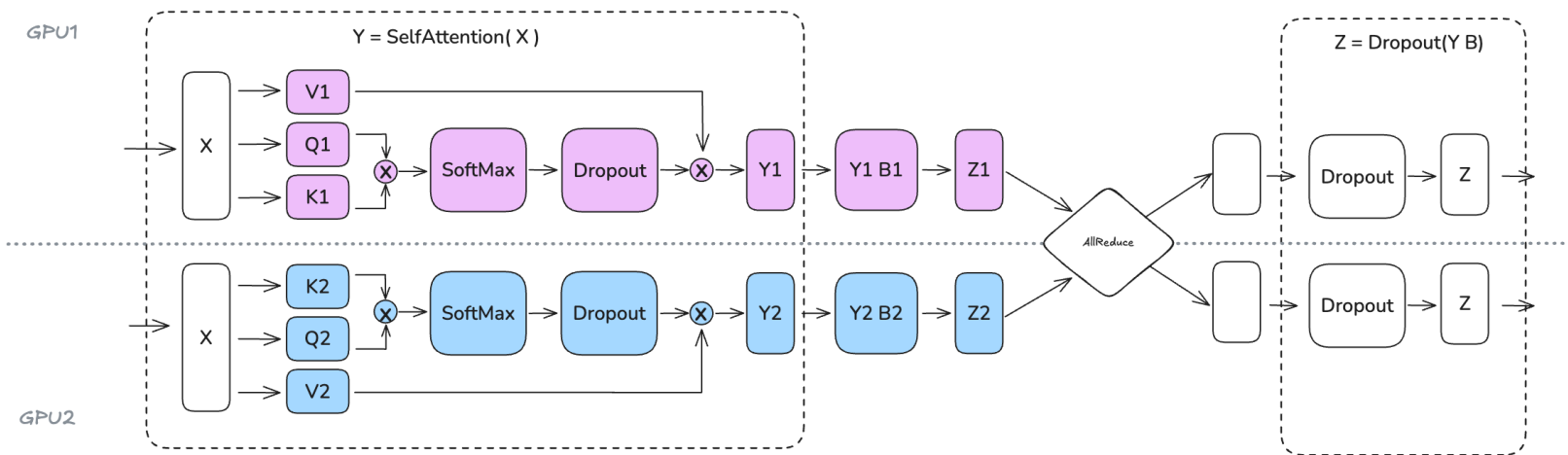
Tensor parallelism splits one matrix product across GPUs

Column split: each rank computes different output features.

Row split: each rank computes a partial sum, then reduce across ranks.



Megatron pairs column and row splits inside a Transformer block

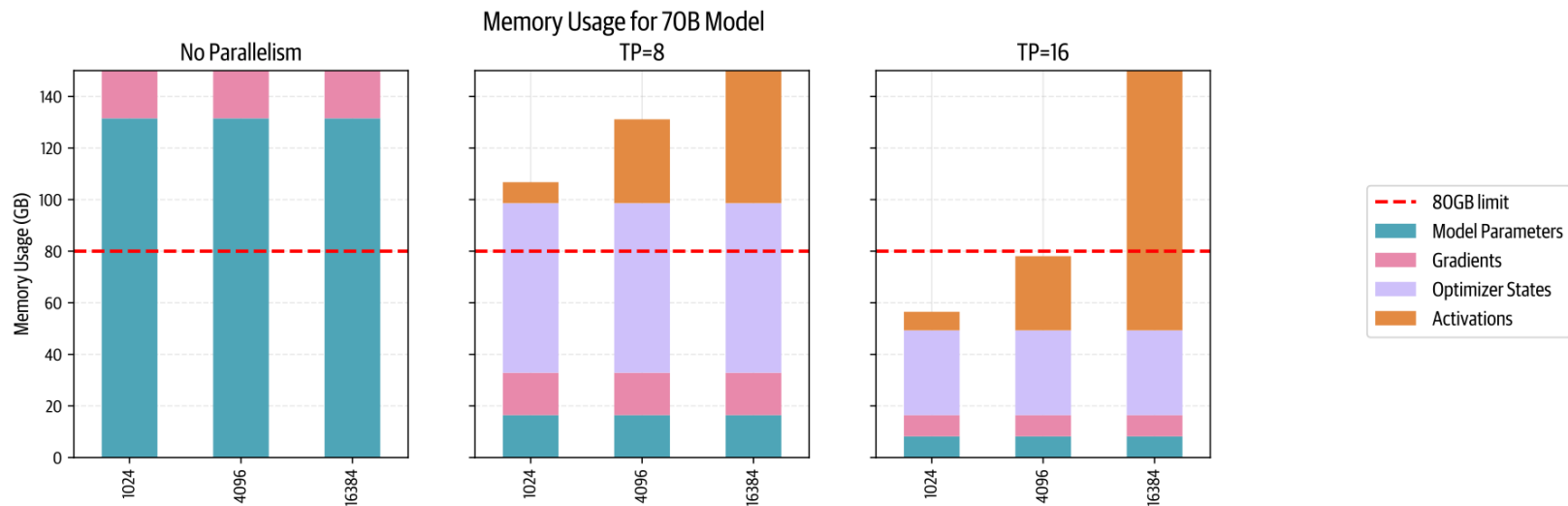


The MLP and attention projections choose shard directions to avoid an all-gather between the two linear layers.

Collectives remain at block boundaries.



TP cuts weights per GPU but communicates activations often

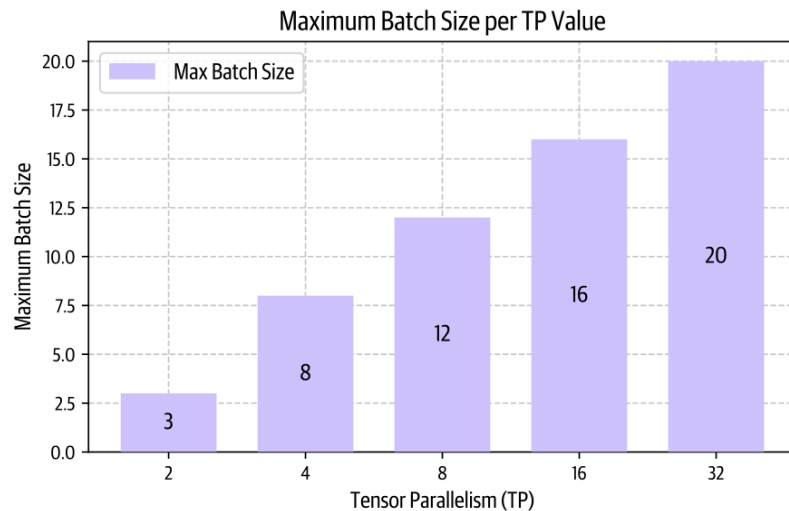
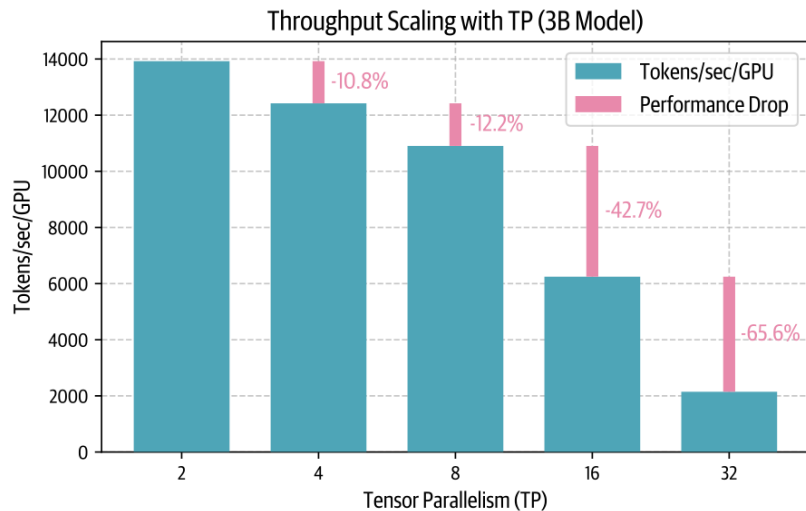


Activation collective volume scales with $B \times S \times H$ per layer.

Fast local links and sufficiently large GEMMs make TP worthwhile.



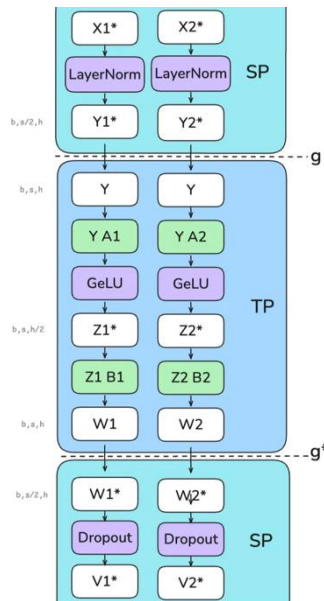
The TP sweet spot is finite, even with fast links



Sequence parallelism shards cheap activations along tokens

LayerNorm, dropout and residual portions need not be fully replicated on every TP rank.

Reduce-scatter/all-gather exchange more structured data for activation memory.



08

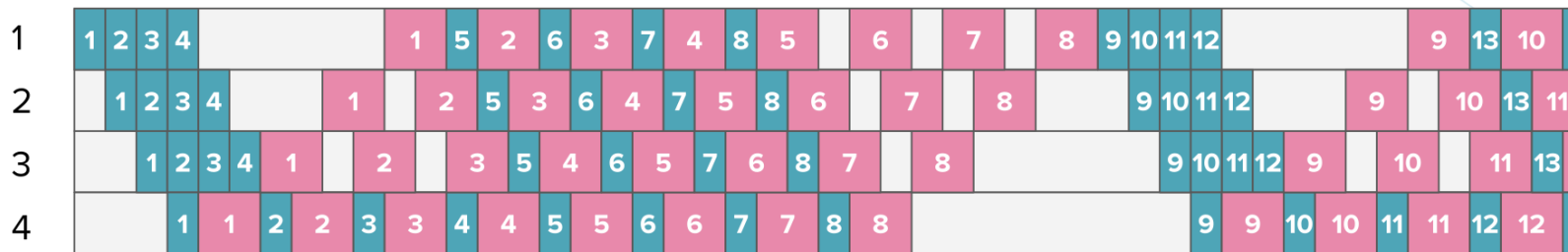
Other axes

Pipeline, context and experts



Pipeline parallelism puts different layers on different GPUs

GPU



Time →

Forward pass

Backward pass

Device idle

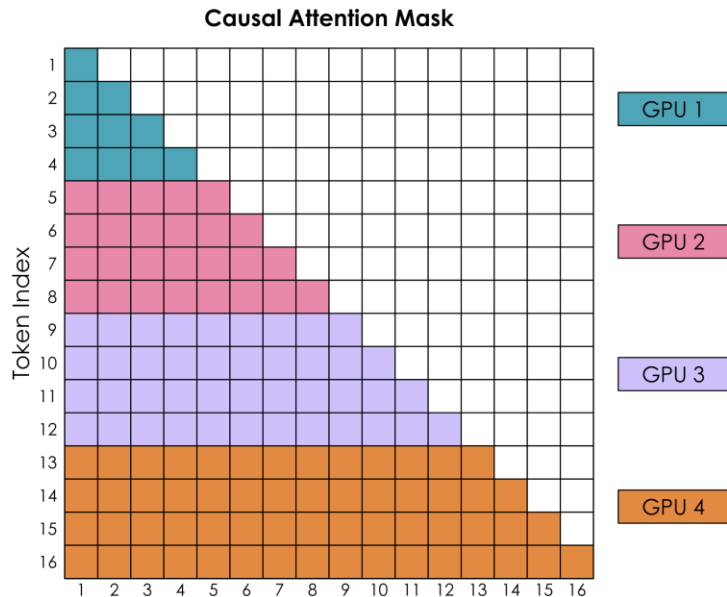
Microbatches fill the pipeline; warm-up and drain create bubbles.

More stages make model fit, but increase scheduling complexity and latency.

Context parallelism splits long sequences

Each rank holds fewer token activations; attention must exchange K/V or partial results.

It becomes attractive when sequence length makes activations or attention impossible locally.



Expert parallelism routes tokens to remote MLP experts

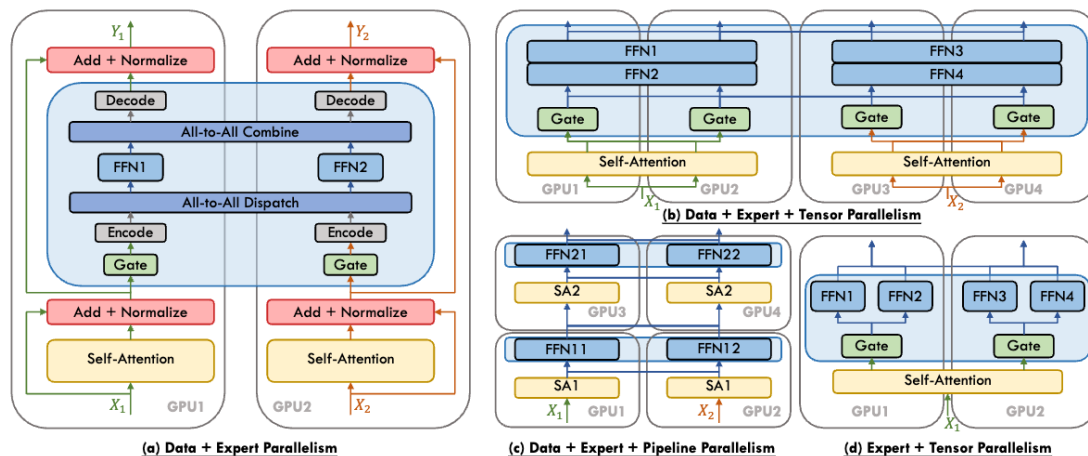


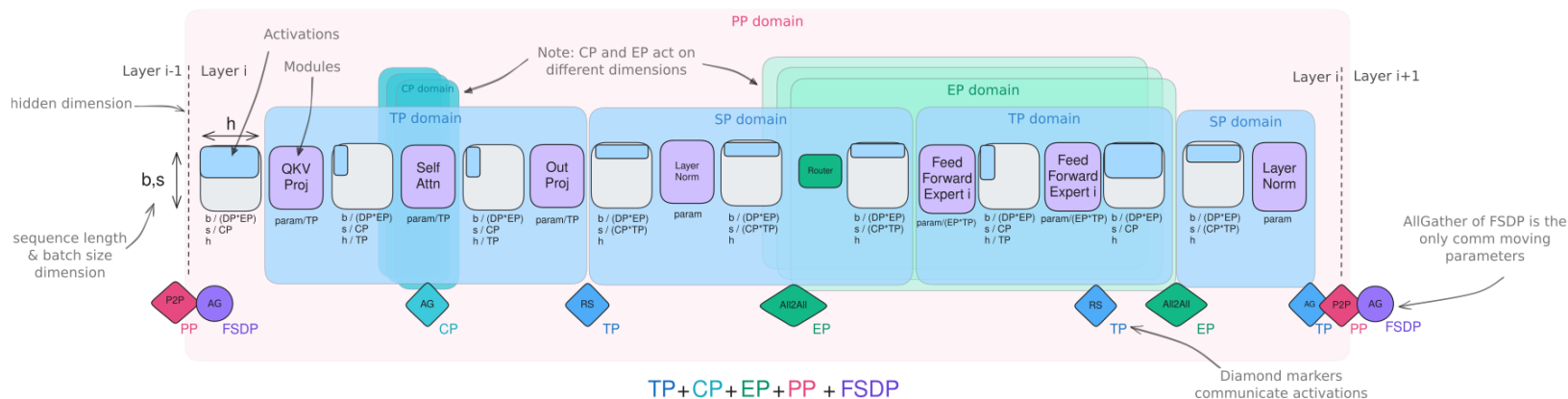
Fig. 8. Schematic depiction of diverse parallel strategies for MoE. For clarity and conciseness, this illustration omits some All-to-All, All-Reduce, Point-to-Point communication within parallelism, and Normalization, Encode, Decode, Gate in subfigures (b), (c), and (d).

MoE activates a subset of parameters per token. Token dispatch/combine is all-to-all.

Load balance, capacity and network topology determine realized gain.



Parallel degrees multiply; bottlenecks do not disappear



Total GPUs = $DP \times TP \times PP \times CP \times EP$ (when axes are orthogonal).

Map frequent TP traffic to fast links; scale DP outward; use PP/CP/EP when model demands it.



09

• Choose a configuration



Case study: eight GPUs, model fits after ZeRO-1

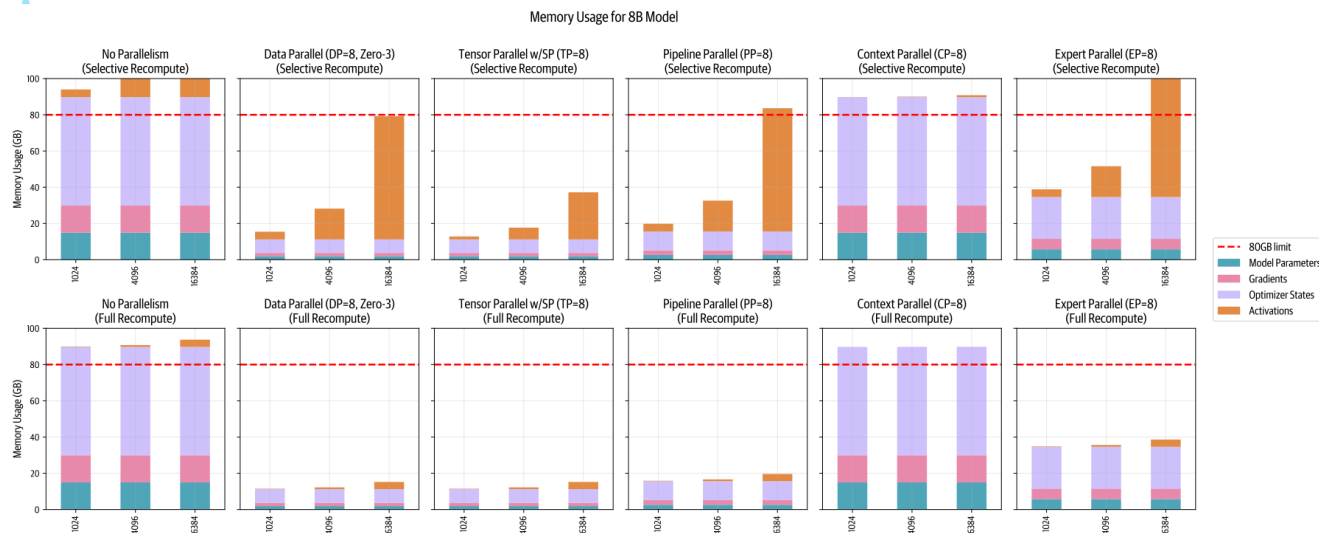
Start with DP + ZeRO-1; measure tokens/s and memory peak.

Introduce TP only if weights/activations or microbatch shapes justify it.

Re-profile after every change; do not add five axes at once.



Case study: model does not fit even with full sharding

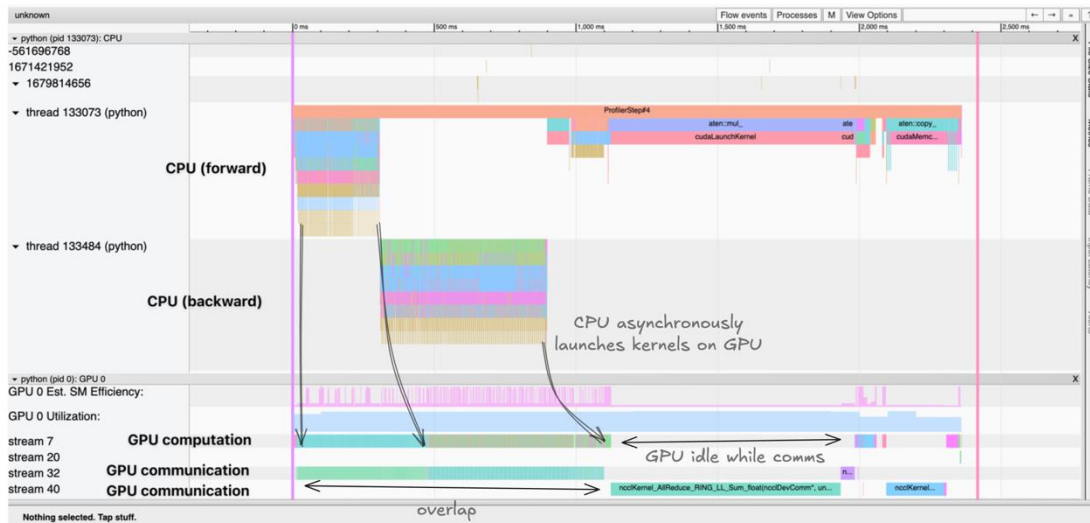


Partition compute and weights with TP/PP; choose topology before degrees.

For extreme context add CP; for MoE add EP and balance token routing.



A sensible tuning loop starts with a measured baseline



A faster training run follows the bytes and the gradients

GPU hierarchy → roofline → measured step → memory lifetime → DP/ZeRO → TP → other axes.

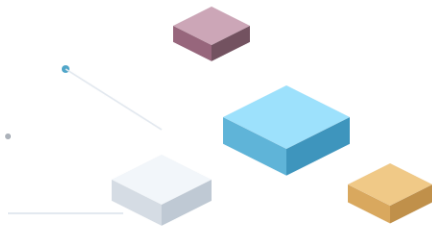
Three questions: what is slow, where does it live, what moves if we split it?



Homework 4: build a causal LM training pipeline

Data + collator → causal LM objective → PyTorch training loop.

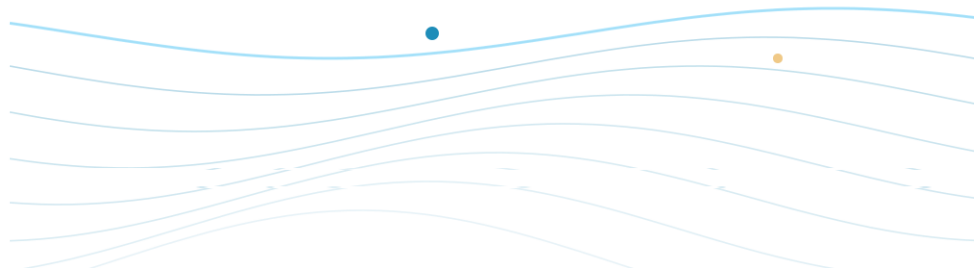
Reproduce it with Hugging Face Trainer. Control microbatch and gradient accumulation.



Homework 4: compare the updates, then report the ablation

Set parameter groups; inspect update geometry. Compare AdamW and Muon at fixed setup.

Vary microbatch/accumulation and submit an ablation with a short report.



Lecture 5: SFT, Alignment & ICL

seminar

today, 15:30

materials

course repository

homework

two weeks, release today

